

 AVALIAÇÃO

Nome:		R.A.:	
Curso:	Ciência da Computação	Data:	____/____/2026
Disciplina:	Estrutura de Dados	Docente:	Prof. Me. Karan Luciano
Avaliação:	☒ N1 ☐ N2 ☐ Sub ☐ Exame	Nota:	

ANTES DE INICIAR A AVALIAÇÃO, LEIA ATENTAMENTE AS ORIENTAÇÕES ABAIXO:**Composição e Distribuição da Pontuação (Total da Avaliação Escrita: 30,0 pontos):**

- **10 Questões Objetivas (Múltipla Escolha):** valor de **1,5 ponto** cada (totalizando **15,0 pontos**).
- **01 Questão de Análise de Código e Alocação Dinâmica:** valor de **7,5 pontos**.
- **01 Questão Discursiva de Paradigmas de Busca e Notação Big-O:** valor de **7,5 pontos**.

A pontuação total obtida nesta prova escrita (0 a 30 pontos) compõe a etapa N1 somada às atividades práticas (20 pontos), totalizando os 50 pontos da N1.

1. Confira se este caderno contém todas as **12 questões** (10 objetivas numeradas de 01 a 10 e 02 discursivas/código numeradas de 11 e 12);
2. **Não será permitido o uso do telefone celular ou qualquer outro dispositivo eletrônico. Devem ser desligados e guardados;**
3. As questões objetivas contêm apenas uma resposta correta. A marcação de mais de uma resposta invalidará a questão;
4. Transcreva as respostas das questões objetivas no Gabarito abaixo, com caneta azul ou preta e sem rasuras;
5. As respostas das questões discursivas e de código devem ser redigidas à caneta no espaço delimitado;
6. **Não será permitido nenhum tipo de consulta a livros, anotações ou colegas;**
7. O tempo mínimo de permanência em sala de aula é de 30 minutos. Os últimos 03 alunos devem permanecer até o término da prova por estes.

GABARITO — QUESTÕES OBJETIVAS

Marque apenas as questões correspondentes à sua avaliação com um 'X'. Rasuras invalidam a resposta.

01	A	B	C	D	E	06	A	B	C	D	E
02	A	B	C	D	E	07	A	B	C	D	E
03	A	B	C	D	E	08	A	B	C	D	E
04	A	B	C	D	E	09	A	B	C	D	E
05	A	B	C	D	E	10	A	B	C	D	E

CADERNO DE QUESTÕES OBJETIVAS

QUESTÃO 01

Ponteiros e Gestão de Memória RAM (1,5 pt)

Em linguagem C, a memória física (RAM) é estruturada como uma vasta sequência ordenada de bytes numerados. Ao declararmos uma variável escalar e um ponteiro associado a ela, como na instrução `int x = 42; int *ptr = &x;`, qual é a finalidade precípua da variável `ptr` na arquitetura do sistema?

- A) Guardar o endereço de memória física onde a variável referenciada está armazenada.
- B) Duplicar automaticamente o valor de `x` em uma área reservada de backup da memória.
- C) Executar a limpeza e liberação automática do espaço ocupado por `x` no término do bloco.
- D) Armazenar valores numéricos fracionários de ponto flutuante de dupla precisão.
- E) Converter o valor decimal de `x` em um tipo caractere imutável.

QUESTÃO 02

Operadores de Indireção (* e &) (1,5 pt)

Considere a execução sequencial do seguinte trecho de código em linguagem C:

```
int numero = 10;
int *ptr = &numero;
*ptr = 25;
```

Após o processamento dessas três instruções, qual será o valor impresso pela instrução `printf("%d", numero);` e qual mecanismo fundamenta essa alteração?

- A) Imprime **10**, pois ponteiros operam apenas sobre cópias locais isoladas em registradores.
- B) Imprime **25**, pois o operador de desreferenciação (*) altera diretamente a posição de memória original de **numero**.
- C) Imprime o endereço hexadecimal da variável **numero** na pilha de execução (*Stack*).
- D) Imprime **0**, pois a atribuição via ponteiro reinicializa o conteúdo da variável apontada.
- E) Ocorre um erro de violação de acesso (*Segmentation Fault*) por atribuição ilegal de tipo inteiro a ponteiro.

QUESTÃO 03

Aritmética de Ponteiros em Vetores (1,5 pt)

Em uma arquitetura computacional de 64 bits onde o tipo de dado `int` ocupa exatamente 4 bytes de memória, um desenvolvedor declara o vetor `int v[5] = {10, 20, 30, 40, 50};`. Sabendo que o início do vetor reside no endereço hexadecimal `0x1000`, a expressão aritmética `*(v + 3)` irá acessar:

- A) O endereço físico `0x1003`, retornando o valor 30.
- B) O endereço físico `0x100C` (deslocamento de 3×4 bytes), recuperando o valor 40.
- C) O elemento de índice 2 do vetor, resultando no valor numérico 30.
- D) Um erro em tempo de compilação, pois identificadores de vetores não admitem operações aritméticas.
- E) O endereço base `0x1000`, somando 3 unidades diretamente ao valor 10 e resultando em 13.

QUESTÃO 04

Alocação Dinâmica na Heap com malloc() (1,5 pt)

Ao construir estruturas de dados que necessitam crescer ou encolher durante o tempo de execução do programa (como listas encadeadas e árvores), a alocação estática na pilha (*Stack*) torna-se insuficiente. Para requisitar blocos contíguos de memória sob demanda diretamente da *Heap*, qual função padrão da biblioteca `<stdlib.h>` deve ser invocada?

- A) `malloc()`, especificando a quantidade necessária de bytes via operador `sizeof`.
- B) `sizeof()`, encarregada de requisitar e inicializar blocos na memória dinâmica.
- C) `printf()`, que aloca buffers dinâmicos de saída padrão.
- D) `system()`, que solicita ao sistema operacional a criação de uma nova partição de memória.
- E) `realloc()`, a única permitida para criar ponteiros dinâmicos a partir do zero.

QUESTÃO 05*Ciclo de Vida da Memória e Memory Leak (1,5 pt)*

Durante a execução contínua de um serviço corporativo de processamento de dados em C, observou-se um consumo crescente da memória RAM da máquina servidora, culminando na interrupção abrupta do processo pelo sistema operacional (*Out of Memory Killer*). A auditoria identificou blocos instanciados repetidamente com `malloc()` sem a devida chamada a `free()`. Esse cenário caracteriza formalmente:

- A) Falha de segmentação (*Segmentation Fault*) induzida pela destruição da tabela de símbolos.
- B) Um vazamento de memória (*Memory Leak*), pois a memória na *Heap* permanece reservada e inacessível pelo encerramento dos ponteiros locais.
- C) Atuação correta do Coletor de Lixo (*Garbage Collector*) nativo do C, que compacta blocos ociosos.
- D) Transbordamento de pilha (*Stack Overflow*) decorrente de chamadas recursivas infinitas.
- E) Corrupção instantânea dos registradores da CPU por ausência do comando `return`.

QUESTÃO 06*Registros Heterogêneos (Structs) e Operador Seta (→) (1,5 pt)*

Considere a declaração de um nó típico de estrutura dinâmica em linguagem C:

```
typedef struct No {  
    int valor;  
    struct No *proximo;  
} No;  
  
No *ponteiro_no = (No*) malloc(sizeof(No));
```

Para atribuir o número 100 ao campo `valor` através do ponteiro `ponteiro_no`, qual sintaxe em conformidade com o padrão ANSI C deve ser adotada?

- A) `ponteiro_no.valor = 100;`
- B) `ponteiro_no→valor = 100;` ou de forma equivalente `(*ponteiro_no).valor = 100;`
- C) `ponteiro_no::valor = 100;`
- D) `&ponteiro_no→valor = 100;`
- E) `*ponteiro_no→valor = 100;`

QUESTÃO 07*Pré-requisito e Mecânica da Busca Binária (1,5 pt)*

A Busca Binária é um algoritmo de alta eficiência amplamente empregado na recuperação de informações em coleções computacionais. Contudo, diferentemente da Busca Sequencial (Linear), ela impõe uma pré-condição mandatória sobre a estrutura de dados para garantir sua corretude. Essa pré-condição consiste em:

- A) A coleção de dados deve possuir obrigatoriamente um número par de elementos.
- B) Os elementos devem estar previamente organizados em ordem monotônica (estritamente ordenados de forma crescente ou decrescente).
- C) A estrutura deve ser obrigatoriamente alocada de forma dinâmica por meio de nós e ponteiros.
- D) O vetor não pode conter elementos repetidos ou valores numéricos nulos.
- E) A busca precisa ser executada obrigatoriamente por múltiplos núcleos de processamento em paralelo.

QUESTÃO 08*Eficiência Assintótica da Busca Binária e Big-O (1,5 pt)*

Em uma base cadastral contendo aproximadamente 1.000.000 (um milhão) de registros devidamente ordenados, um algoritmo de busca sequencial pode requerer, no pior caso, até 1.000.000 de comparações para encontrar uma chave ou constatar sua ausência $O(n)$. Em contrapartida, a Busca Binária resolve a mesma tarefa com no máximo 20 comparações. Essa superioridade deve-se ao fato de que:

- A) A Busca Binária possui complexidade de tempo logarítmica $O(\log_2 n)$, reduzindo pela metade o espaço de busca a cada iteração.
- B) A Busca Binária acessa diretamente a posição exata por cálculo de dispersão (*hash*) em tempo constante $O(1)$.
- C) O algoritmo transforma o vetor linear em uma matriz multidimensional para executar buscas diagonais.
- D) Ela realiza comparações simultâneas em todos os índices utilizando recursos da Unidade de Ponto Flutuante (FPU).
- E) A Busca Binária ignora os valores intermediários e compara apenas o elemento inicial e o final.

QUESTÃO 09*Mecânica do Bubble Sort e Pior Caso (1,5 pt)*

O *Bubble Sort* (Método da Bolha) é um dos algoritmos de ordenação elementares mais conhecidos. Sobre sua mecânica de funcionamento e comportamento assintótico, assinale a proposição correta:

- A) Opera selecionando repetidamente o menor item da partição não ordenada e posicionando-o no início do vetor.
- B) Percorre a coleção comparando pares de elementos adjacentes e efetuando trocas quando fora de ordem, apresentando complexidade de tempo de pior caso $O(n^2)$.
- C) Divide iterativamente o conjunto ao meio até atingir partições atômicas unitárias, garantindo tempo $O(n \log n)$ em todos os casos.
- D) Consegue ordenar vetores estritamente decrescentes efetuando apenas uma única passada linear $O(n)$.
- E) Necessita alocar um vetor auxiliar secundário com o mesmo tamanho da entrada para consolidar os dados.

QUESTÃO 10*Paradigma de Divisão e Conquista no Merge Sort (1,5 pt)*

O algoritmo *Merge Sort* destaca-se pela estabilidade e previsibilidade de desempenho, garantindo complexidade assintótica $O(n \log_2 n)$ em todos os seus cenários (melhor, médio e pior caso). O princípio metodológico que rege a estrutura do Merge Sort baseia-se em:

- A) Paradigma Guloso (*Greedy*), que seleciona a menor decisão imediata em cada passo sem possibilidade de retrocesso.
- B) Divisão e Conquista, decompondo recursivamente o problema em subproblemas menores até atingir casos triviais, combinando-os subsequentemente de forma ordenada (intercalação).
- C) Busca Exaustiva com Retrocesso (*Backtracking*), testando todas as permutações possíveis dos itens até encontrar a configuração ordenada.
- D) Programação Dinâmica com memoização estrita em matriz bidimensional de adjacências.
- E) Trocas aleatórias estocásticas baseadas no método de Monte Carlo.

QUESTÕES DISCURSIVAS E ANÁLISE DE CÓDIGO

QUESTÃO 11 (ANÁLISE DE CÓDIGO)*Encadeamento Dinâmico, Nós e Ciclo de Vida na Heap (7,5 pts)*

Considere a rotina de manipulação de nós dinâmicos em C apresentada a seguir, desenvolvida para criar uma pequena lista simplesmente encadeada na memória *Heap*:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int info;
    struct Node *next;
} Node;

int main() {
    Node *n1 = (Node*) malloc(sizeof(Node));
    Node *n2 = (Node*) malloc(sizeof(Node));

    n1->info = 10;
    n1->next = n2;

    n2->info = 20;
    n2->next = NULL;

    // Trecho sob auditoria
    Node *p = n1;
    while (p != NULL) {
        printf("[%d] → ", p->info);
        p = p->next;
    }
    printf("NULL ");

    free(n1);
    // Linha final do desenvolvedor:
    n2 = NULL;
    return 0;
}
```

a) (3,75 pts) Qual é a **saída exata** impressa no console pela execução do laço **while**? Explique detalhadamente como os ponteiros **n1**, **n2** e o campo **next** conectam logicamente esses dois blocos isolados na memória RAM.

b) (3,75 pts) Aponte explicitamente a **falha crítica de gerenciamento de memória** cometida nas linhas finais do programa (**free(n1); n2 = NULL;**). Identifique qual patologia de memória ocorre (*Memory Leak* ou *Dangling Pointer*), em qual bloco ela ocorre e apresente a correção adequada em código C para liberar ambos os nós sem resíduos.

QUESTÃO 12 (DISCURSIVA)

Paradigmas de Busca e Análise de Complexidade Assintótica (7,5 pts)

Uma empresa de transporte rodoviário monitora o rastreamento em tempo real de encomendas em uma base cadastral contendo **500.000 pacotes** registrados. O engenheiro de software da equipe deve escolher entre a implementação de um algoritmo de **Busca Sequencial (Linear)** e um algoritmo de **Busca Binária** para atender às consultas concorrentes efetuadas pelos usuários no aplicativo.

a) (4,50 pts) Preencha o quadro comparativo analítico abaixo, contrastando rigorosamente o pré-requisito de ordenação, a quantidade máxima de comparações no pior caso (expressa em Notação Big-O) e a principal característica/vantagem de cada método:

Critério de Análise	Busca Sequencial (Linear)	Busca Binária
Pré-requisito dos dados		
Complexidade (Pior Caso)		
Cenário de uso recomendado		

b) (3,00 pts) Suponha que a coleção receba dezenas de novas encomendas a cada minuto, inseridas de forma desordenada no final da estrutura. Discuta tecnicamente o impacto do custo de reordenação prévia ($O(n \log n)$) sobre a viabilidade econômica e computacional do uso contínuo da Busca Binária nesse cenário altamente dinâmico, contrastando com a inserção $O(1)$ da Busca Linear.